

Preguntas Multiple Choice: **Fundamentos de Organización de Datos (Archivos).**

1. A partir de un archivo con registros de longitud fija y luego de algunas operaciones con el mismo:

- a. Nunca genera fragmentación.
- b. Puede generar fragmentación interna.
- c. Puede generar fragmentación externa.
- d. Las últimas dos son correctas.*

2. A partir del NRR:

- a. Se puede acceder a un registro de un archivo fragmentado en un solo acceso.
- b. Se puede acceder a un registro de un archivo no fragmentado en un solo acceso.
- c. Se puede acceder a un registro de un archivo en un sólo acceso.
- d. Se puede lograr acceso directo a un archivo.
- e. Todas las anteriores.*
- f. Algunas de las anteriores.

3. ¿Cuál de las siguientes definiciones corresponde a archivo?:

- a. Colección de registros que abarca un conjunto de entidades con ciertos aspectos en común y organizados para un propósito particular.
- b. Colección de registros semejantes almacenados en disco rígido.
- c. Colección de registros del mismo tipo almacenados en un dispositivo de memoria secundaria.
- d. *Todas las anteriores.*

4. ¿Cuál de las siguientes definiciones NO se corresponde con un índice?:

- a. Es una estructura de datos adicional que permite agilizar el acceso a la información almacenada en un archivo.
- b. Es una estructura de datos adicional que permite agilizar el recorrido ordenado de un archivo.
- c. *Es una estructura adicional implementada con un árbol balanceado que permite agilizar el acceso a la información almacenada en un archivo.*
- d. *Es una estructura auxiliar que permite ordenar físicamente un archivo, así se pueden hacer búsquedas más eficientes.*
- e. Todas las anteriores.
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

5. Con respecto a los buffers de E/S:

- a. *Ocupan lugar en memoria RAM.*
- b. *El SO está encargado de manipularlos.*
- c. *Mejoran la performance en lecturas y escrituras.*
- d. Ninguna de las anteriores.

6. Con respecto a los índices:

- a. Al realizar bajas lógicas sobre un índice primario, es posible recuperar esos espacios con nuevas altas.
- b. Un índice es una estructura de datos (adicional al archivo de datos) que debe utilizar registros de longitud variable.
- c. *Un índice permite imponer orden en un archivo de datos, sin que éste realmente se reacomode.*
- d. Ninguna de las anteriores.

7. Con respecto a un archivo con registros de longitud fija: ¿Cuáles de estas alternativas son correctas?

- a. *Utilizar NRR para tener acceso directo.*
- b. Utilizar un indicador de longitud al inicio de cada registro.
- c. Utilizar un segundo archivo con la información de la dirección del byte de inicio de cada registro.
- d. Ninguna de las anteriores.

8. Dado un archivo:

- a. Siempre se necesita tener un índice asociado.
- b. Un índice asociado le permite optimizar las operaciones de alta.
- c. Siempre debe estar ordenado.
- d. Ninguna de las anteriores.**

9. Dado un archivo con 1000 registros:

- a. Siempre se puede llevar a memoria RAM para hacer búsquedas más eficientes.
- b. No siempre se puede llevar a memoria RAM para hacer búsquedas más eficientes.**
- c. Siempre se puede realizar búsqueda dicotómica.
- d. No puede realizarse búsqueda dicotómica.
- e. Algunas de las anteriores.

10. Dado un archivo de índice secundario implementado con el método de listas invertidas:

- a. Es posible asociar sólo una cantidad acotada de claves primarias.
- b. En ocasiones se desperdicia espacio, ya que se debe reservar el mismo.
- c. El método consiste en usar un archivo adicional de las claves primarias que son referenciadas desde el índice secundario.**
- d. Ninguna de las anteriores.

11. El acceso promedio para recuperar un dato en un archivo desordenado:

- a. Puede tener orden lineal.
- b. Puede tener orden logarítmico.
- c. Tiene orden constante (uno).
- d. a y b son correctas.
- e. *Ninguna de las anteriores.*

12. El acceso promedio para recuperar un dato en un archivo desordenado:

- a. *Tiene orden lineal.*
- b. Tiene orden logarítmico.
- c. Tiene orden constante (uno).
- d. a y b son correctas.
- e. Ninguna de las anteriores.

13. El acceso secuencial a un archivo es:

- a. *Acceso a los registros uno tras otro y en el orden físico en el que están guardados.*
- b. Acceso a los registros de acuerdo al orden establecido por otra estructura.
- c. Acceso a un registro determinado sin necesidad de haber accedido a los predecesores.
- d. Ninguna de las anteriores.

14. El algoritmo de actualización maestro-detalles:

- a. Se puede implementar si los archivos están ordenados.
- b. Se puede implementar con el archivo ordenado.
- c. Se puede implementar con los archivos detalles ordenados.
- d. Se puede implementar si los archivos están desordenados.
- e. *Todas las anteriores.*
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

15. El concepto de fragmentación externa en un archivo:

- a. Se da sólo en registros de longitud fija.
- b. *Se da sólo en registros de longitud variable.*
- c. Se da en archivos ordenados de longitud variable.
- d. Se debe analizar sólo en archivos ordenados de longitud fija.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

16. El concepto de fragmentación interna en un archivo:

- a. Se puede dar en registros de longitud fija.
- b. Se puede dar en registros de longitud variable.
- c. Se puede dar en archivos ordenados de longitud variable.
- d. Se puede analizar en archivos ordenados de longitud fija.
- e. *Todas las anteriores.*
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

17. El concepto de fragmentación interna en un archivo:

- a. Se puede dar sólo en registros de longitud fija.
- b. Se puede dar sólo en registros de longitud variable.
- c. Se puede dar sólo en archivos ordenados de longitud variable.
- d. Se puede analizar sólo en archivo ordenados de longitud fija.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

18. El NRR puede utilizarse:

- a. *En archivos con registros de longitud fija.*
- b. En archivos con registros de longitud variable.
- c. *Para realizar acceso directo a un registro.*
- d. Para realizar acceso secuencial en un archivo.

19. El procedimiento de alta de información en un archivo:

- a. Siempre agrega información al final del archivo.
- b. Puede recuperar espacio dado de baja físicamente.
- c. Siempre recupera espacio dado de baja lógicamente.
- d. *Ninguna de las anteriores.*

20. El proceso de alta de registro con recuperación de espacio:

- a. Se debe realizar con registros de longitud variable.
- b. Se debe realizar con registros de longitud fija.
- c. Se realiza sólo con registros de longitud fija.
- d. Se realiza sólo con registros de longitud variable.
- e. *Ninguna de las anteriores.*

21. El proceso de alta de registro por ajuste óptimo:

- a. Se puede realizar con registros de longitud fija.
- b. Se debe realizar con registros de longitud fija.
- c. Se puede realizar con registros de longitud variable.
- d. Se debe realizar con registros de longitud variable.
- e. *Ninguna de las anteriores.*

22. El proceso de alta de registro por peor ajuste:

- a. Se puede realizar con registros de longitud fija.
- b. Se debe realizar con registros de longitud fija.
- c. Se puede realizar con registros de longitud variable.
- d. *Se debe realizar con registros de longitud variable.*
- e. Ninguna de las anteriores.

23. El proceso de baja lógica (sin ningún agregado de otras operaciones) de un archivo:

- a. *Nunca recupera espacio en disco.*
- b. Siempre recupera espacio en disco.
- c. A veces, recupera espacio en disco.
- d. No dispongo de información suficiente para responder la pregunta.

24. El proceso de baja en un archivo con registros de longitud variable:

- a. Puede recuperar el espacio disponible con nuevas altas.
- b. Puede recuperar el espacio disponible compactando periódicamente el archivo.
- c. Puede recuperar el espacio disponible compactando el archivo ante cada baja.
- d. *Todas las anteriores.*

25. El proceso de baja lógica:

- a. *Está diseñado para borrar un registro de un archivo.*
- b. Necesita que el archivo esté ordenado.
- c. Necesita que el archivo esté desordenado.
- d. Se aplica sólo a archivos con registros con longitud fija.
- e. Se aplica sólo a archivos con registros con longitud variable.
- f. Todas las anteriores.
- g. Alguna de las anteriores.

26. El proceso de baja lógica:

- a. *Está diseñado para borrar un registro de un archivo.*

- b. No necesita que el archivo esté ordenado.*
- c. Necesita que el archivo esté desordenado.
- d. Se aplica sólo a archivos con registros con longitud fija.
- e. Se aplica sólo a archivos con registros con longitud variable.
- f. Todas las anteriores.
- g. Algunas de las anteriores.*

27. El proceso de baja lógica:

- a. Se puede aplicar a archivos desordenados.
- b. Se puede aplicar a archivos ordenados.
- c. Se puede aplicar a archivos de longitud fija.
- d. Se puede aplicar a archivos de longitud variable.
- e. Se puede aplicar a cualquier archivo.
- f. No se puede aplicar si el archivo está vacío.
- g. Todas las anteriores.*

28. El proceso de compactación de un archivo tiene sentido ser aplicado:

- a. Luego de realizar una operación de alta.
- b. Luego de realizar una operación de baja lógica.*
- c. Luego de realizar una operación de baja físicamente.
- d. Luego de realizar una operación de modificación.

29. El proceso de corte de control:

- a. Actualiza el archivo maestro a partir de un archivo detalle
- b. Actualiza el archivo maestro a partir de varios archivos detalle.
- c. Genera un único archivo, uniendo un archivo maestro con un archivo detalle.
- d. Genera un único archivo, uniendo un archivo maestro con todos los archivos detalle.
- e. Ninguna de las anteriores.*

30. El proceso de merge de archivos:

- a. Requiere que todos los archivos estén ordenados.
- b. Requiere que todos los archivos estén ordenados por el mismo criterio.
- c. Puede hacerse **con/sin** los archivos ordenados.*
- d. No puede realizarse sin los archivos ordenados.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

31. En el caso de realizar una alta de un registro:

- a. Se puede reaprovechar con la técnica de primer ajuste algún espacio de registro dado de baja previamente.
- b. Se puede reaprovechar algún espacio de registro dado de baja previamente.
- c. Puede insertarse al final del archivo.
- d. Se debe insertar en un lugar donde el registro quepa.
- e. Todas las anteriores.*
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

32. En el caso de realizar una alta de un registro:

- a. Se puede reaprovechar algún espacio de registro dado de baja previamente.
- b. Se debe reaprovechar algún espacio de registro dado de baja previamente.
- c. Se puede reaprovechar si los registros son de longitud fija.
- d. Se puede reaprovechar si los registros son de longitud variable.
- e. Algunas de las anteriores.*
- f. Ninguna de las anteriores.

33. En un archivo con registros de longitud variable:

- a. *Puede utilizar "\$" como delimitador de fin de registro.*
- b. Cuando un registro se modifica utiliza el mismo espacio.
- c. Se puede utilizar cualquier lugar libre de un archivo para insertar un registro.
- d. Siempre se utiliza la política de mejor ajuste para recuperar espacio.
- e. Algunas de las anteriores.

34. En un índice secundario:

- a. Encontrar un registro es, a veces, más lento que sobre un índice primario.
- b. Encontrar un registro es igual de rápido que sobre un índice primario.
- c. Encontrar un registro puede ser más rápido que sobre un índice primario.
- d. Si la clave a buscar no se repitiera puede ser igual de rápida su búsqueda que en un índice unívoco.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

35. Es posible aplicar la búsqueda binaria en:

- a. Archivos desordenados con registros de longitud variable.
- b. Archivos desordenados con registros de longitud fija.
- c. Archivos ordenados con registros de longitud variable.
- d. *Archivos ordenados con registros de longitud fija.*
- e. Ninguna de las anteriores.

36. La búsqueda binaria es aplicable a:

- a. Archivos con registros de longitud variable.
- b. Archivos desordenados con registros de longitud fija.
- c. Archivos ordenados con registros de longitud fija.**
- d. Ninguna de las anteriores.

37. La eficacia de un algoritmo de búsqueda de registros en un archivo de datos que no está ordenado:

- a. Orden constante.
- b. Orden lineal.**
- c. Orden logarítmico.
- d. No tengo suficientes datos para responder.
- e. Ninguna de las anteriores.

38. La eficiencia promedio de búsqueda en un archivo sin orden es:

- a. Orden lineal.**
- b. Orden logarítmico.
- c. 1.
- d. No dispongo datos para contestar la pregunta.

39. La eliminación lógica de elementos de un archivo:

- a. Es menos eficiente en términos de espacio que la física.**
- b. Es menos eficiente en términos de tiempo de respuesta del algoritmo que la física.
- c. Es más eficiente en términos de espacio y de tiempo de respuesta del algoritmo que la física.
- d. Algunas de las anteriores.
- e. Ninguna de las anteriores.

40. La fragmentación en un archivo de longitud fija:

- a. Dificulta la baja física.
- b. Dificulta la baja lógica.
- c. Produce pérdida de espacio.**
- d. Es menor utilizando primer ajuste.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

41. La función EOF:

- a. Puede devolver verdadero después de un reset.
- b. Puede devolver falso después de una lectura.
- c. Puede devolver verdadero después de una lectura.
- d. Devuelve verdadero si estas al final del archivo.
- e. Devuelve falso si no estas al final del archivo.
- f. Todas las anteriores.**
- g. Algunas de las anteriores.
- h. Ninguna de las anteriores.

42. La operación assign:

- a. Se utiliza para abrir un archivo.
- b. Se utiliza para posicionarse en el primer registro del archivo.
- c. Vincula el archivo lógico con el archivo físico.**
- d. Se utiliza para saber la longitud del archivo.
- e. Algunas de las anteriores.

43. La operación RESET():

- a. Se realizar luego de la operación REWRITE().

- b. Se realiza previo a la operación ASSIGN().
- c. Abre un archivo para leer o escribir.
- d. Abre un archivo sólo para escribir.
- e. *Abre un archivo.*

44. La operación REWRITE():

- a. Se realiza luego de la operación RESET().
- b. Se realiza previo a la operación ASSIGN().
- c. Abre un archivo para leer o escribir.
- d. Abre un archivo sólo para leer.
- e. *Abre un archivo.*

45. La operación seek en un archivo:

- a. Permite posicionarse directamente al final del archivo.
- b. Permite posicionarse directamente en el primer registro del archivo.
- c. Permite conocer la posición actual dentro del archivo.
- d. *Ninguna de las anteriores.*

46. La política de primer ajuste, que permite recuperar espacio borrado en un archivo:

- a. Sólo se aplica en registros de longitud fija.
- b. *Sólo se aplica en registros de longitud variable.*
- c. Genera fragmentación interna.
- d. Genera fragmentación externa.
- e. Algunas de las anteriores.

47. La política de recuperación de espacio de mejor ajuste, en archivos con registros de longitud fija:

- a. A veces, genera fragmentación externa.
- b. Siempre genera fragmentación externa.
- c. A veces, genera fragmentación interna.
- d. Siempre genera fragmentación interna.
- e. *No corresponde utilizarla con archivos de longitud fija.*

48. La política de recuperación de espacio de peor ajuste, en un archivo con registros de longitud fija:

- a. Puede generar fragmentación interna.
- b. Genera siempre fragmentación interna.
- c. Puede generar fragmentación externa.
- d. Genera siempre fragmentación externa.
- e. *No corresponde utilizarla con archivos de longitud fija.*

49. La técnica de altas reutilizando espacio borrado conocida como mejor ajuste en archivos de longitud fija:

- a. Asigna al registro en el primer espacio que encuentra donde quepa.
- b. Asigna el registro en el espacio donde quepa, de tamaño menor.
- c. Asigna el registro en el espacio donde quepa, de tamaño mayor.
- d. Algunas de las anteriores.
- e. *Ninguna de las anteriores.*

50. La técnica de mejor ajuste:

- a. *Asigna un registro nuevo (alta) en una posición que quepa de tamaño menor.*
- b. Asigna un registro nuevo (alta) en una posición que quepa de tamaño mayor.
- c. *Asigna el registro al final del archivo si no hay lugar en posiciones intermedias.*
- d. Todas las anteriores.
- e. *Algunas de las anteriores.*

f. Ninguna de las anteriores.

51. La técnica de mejor ajuste:

- a. Permite borrar elementos de un archivo que contiene registros de longitud fija.
- b. Permite borrar elementos de un archivo que contiene registros de longitud variable.
- c. Permite hacer baja lógica.
- d. Permite hacer baja física.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.**

52. La técnica de peor ajuste:

- a. Asigna un registro nuevo en la posición que quepa de tamaño menor.
- b. Asigna un registro nuevo en la posición que quepa de tamaño mayor.**
- c. Asigna el registro al final del archivo si no hay lugar en posiciones intermedias.**
- d. Todas las anteriores.
- e. Algunas de las anteriores.**
- f. Ninguna de las anteriores.

53. La técnica de peor ajuste:

- a. Se aplica a archivos con registros de longitud fija.
- b. Se aplica a archivos con registros de longitud variable.**
- c. Se combina con la técnica de baja física de datos.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

54. La técnica de primer ajuste:

- a. Asigna un registro nuevo (alta) en la posición que quepa de tamaño menor.
- b. Asigna un registro nuevo (alta) en la posición que quepa de tamaño mayor.
- c. Asigna el registro al final del archivo si no hay lugar en posiciones intermedias.**
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

55. La técnica de primer ajuste:

- a. Permite borrar elementos de un archivo que contiene registros de longitud fija.
- b. Permite borrar elementos de un archivo que contiene registros de longitud variable.
- c. Permite hacer baja lógica.
- d. Permite hacer baja física.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.**

56. La técnica de primer ajuste:

- a. Se aplica a archivos con registros de longitud fija.
- b. Se aplica a archivos con registros de longitud variable.**
- c. Se combina con la técnica de baja física de datos.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

57. Los archivos con registros de longitud variable:

- a. Ocupan menos espacio que los archivos con registros con longitud fija.
- b. Ocupan más espacio que los archivos con registros con longitud fija.
- c. Ocupan el mismo espacio que los archivos con registros con longitud fija.
- d. Ninguna de las anteriores.**

58. Para actualizar un archivo maestro a partir de N archivos detalles:

- a. *Es necesario que la información que contengan sea compatible.*
- b. Es necesario que todos los archivos estén ordenados.
- c. Es necesario sólo que los archivos detalles estén ordenados.
- d. Es necesario sólo que el archivo maestro esté ordenado.
- e. Es necesario que todos los archivos tengan la misma estructura.

59. Para actualizar un archivo maestro a partir de N archivos detalles:

- a. Es necesario que los archivos detalles estén ordenados.
- b. Es necesario que el archivo maestro esté ordenado.
- c. Es necesario que todos los archivos tengan la misma estructura.
- d. *Ninguna de las anteriores.*

60. Para aplicar un algoritmo de merge:

- a. *Es necesario más de un archivo.*
- b. Es necesario que el archivo maestro esté ordenado.
- c. Es necesario que los detalles estén ordenados.
- d. No es necesario que el archivo maestro esté ordenado.
- e. *No es necesario que los archivos detalles estén ordenados.*
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

61. Para poder realizar merge entre dos archivos:

- a. Los archivos deben estar ordenados con índices implementados con alguna estructura de dato no lineal.
- b. Los archivos deben estar ordenados con índices implementados con alguna estructura de dato lineal.
- c. Los archivos deben estar ordenados.
- d. Los archivos no deben estar ordenados.
- e. *Ninguna de las anteriores.*

62. Para que tenga sentido un algoritmo de corte de control:

- a. El archivo no necesita estar ordenado.
- b. El archivo puede estar ordenado.
- c. El archivo puede estar organizado por dispersión.
- d. El archivo debe estar organizado por dispersión.
- e. El archivo necesita, al menos, un índice asociado.
- f. alguna de las anteriores.
- g. *Ninguna de las anteriores.*

63. Para realizar un algoritmo de actualización maestro-detalles:

- a. *Se requiere, al menos, 2 archivos.*
- b. Se requiere que los archivos estén ordenados.
- c. Se requiere que los archivos estén desordenados.
- d. Se requiere que los archivos tengan la misma estructura.
- e. Se requiere que los archivos tengan la misma estructura.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

64. Un archivo con registros de longitud variable:

- a. Admite una organización mediante hashing.
- b. Admite sólo bajas lógicas.
- c. Admite sólo bajas físicas.
- d. Admite sólo política de mejor ajuste para aprovechamiento de espacio.
- e. *Ninguna de las anteriores.*

65. Un algoritmo de actualización maestro-detalles:

- a. Permite actualizar un archivo maestro con un solo archivo detalle.
- b. Permite presentar la información con un formato especial.
- c. *Permite actualizar un archivo maestro a partir de uno o varios archivos detalle.*
- d. Permite mezclar en un único archivo los registros del archivo maestro y los registros de los archivos de detalle.
- e. Ninguna de las anteriores.

66. Un algoritmo de actualización maestro-detalles:

- a. Requiere que todos los archivos tengan la misma estructura.
- b. *Puede realizarse entre archivos con diferente estructura.*
- c. Requiere que los archivos estén ordenados.
- d. Requiere que los archivos estén desordenados.
- e. Algunas de las anteriores.

67. Un algoritmo de búsqueda en un archivo:

- a. Es más eficiente si el archivo está ordenado.
- b. *Puede ser más eficiente si se considera como precondition que el archivo está ordenado.*
- c. Es igual de eficiente si el archivo está ordenado o desordenado.
- d. Ninguna de las anteriores.

68. Un algoritmo de corte de control:

- a. Permite actualizar un archivo maestro con un archivo detalle.
- b. Permite actualizar un archivo maestro a partir de varios archivos detalle.
- c. *Permite presentar la información con una estructura especial.*
- d. Permite actualizar un archivo con varios archivos detalles.

69. Un algoritmo de corte de control:

- a. *Se debe aplicar a un archivo ordenado para obtener un resultado coherente.*
- b. Se debe aplicar a un archivo desordenado para obtener un resultado coherente.
- c. *Es necesario que el archivo esté ordenado.*
- d. Es suficiente que el archivo esté ordenado por un criterio.
- e. *Algunas de las anteriores.*
- f. Ninguna de las anteriores.

70. Un algoritmo de corte de control:

- a. *Se puede aplicar sobre un archivo con registros de longitud fija.*
- b. *Se puede aplicar sobre un archivo con registros de longitud variable.*
- c. Se aplica sobre un archivo con registros de longitud fija.
- d. Se aplica sobre un archivo con registros de longitud variable.
- e. Todas las anteriores.
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

71. Un archivo con 20 registros:

- a. *No necesita ordenarse.*
- b. Necesita ordenarse.
- c. Si se ordena, debe mantenerse ordenado.

- d. Si se ordena, debe aceptar sólo búsquedas binarias.
- e. Si está desordenado, la búsqueda necesita saber la posición.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

72. Un archivo con registros de longitud fija:

- a. *A veces, tiene fragmentación.*
- b. Debe tener fragmentación.
- c. No puede tener fragmentación.
- d. Ninguna de las anteriores.

73. Un archivo con registros de longitud fija:

- a. Puede tener un delimitador de fin de registro.
- b. Debe tener un delimitador de fin de registro.
- c. Puede tener registros del mismo tamaño.
- d. Puede tener registros con distinto tamaño.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

74. Un archivo con registros de longitud variable:

- a. *Puede estar ordenado por algún criterio.*
- b. Debe estar ordenado por algún criterio.
- c. Nunca puede ordenarse por algún criterio.
- d. *Puede tener un caracter delimitador, por ejemplo &.*
- e. *Algunas de las anteriores.*
- f. Ninguna de las anteriores.

75. Un archivo de datos:

- a. Necesariamente tiene registros de longitud fija.
- b. Necesariamente tiene registros de longitud variable.
- c. Puede tener registros de longitud fija y variable en el mismo archivo.
- d. *Ninguna de las anteriores.*

76. Un archivo directo:

- a. Debe contener registros de longitud fija.
- b. Debe contener registros de longitud variable.
- c. Puede contener registros de longitud fija.
- d. Puede contener registros de longitud variable.
- e. a y b son correctas.
- f. a y c son correctas.
- g. *c y d son correctas.*
- h. b y d son correctas.

77. Un archivo directo:

- a. Permite acceso secuencial únicamente.
- b. Permite acceso secuencial indizado.
- c. *Permite acceso directo.*
- d. a, b y c son correctas.
- e. Ninguna de las anteriores.

78. Un archivo en el cual se accede a un registro luego de acceder a su predecesor en algún orden:

- a. *Puede ser un archivo serie.*
- b. *Puede ser un archivo secuencial.*
- c. Debe ser un archivo serie.

- d. Debe ser un archivo secuencial.
- e. Algunas de las anteriores.*
- f. Ninguna de las anteriores.

79. Un archivo en el cual se accede a un registro luego de acceder a su predecesor en orden físico:

- a. Puede ser un archivo serie.
- b. Puede ser un archivo secuencial.
- c. Debe ser un archivo serie.*
- d. Debe ser un archivo secuencial.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

80. Un archivo físicamente ordenado:

- a. Es más fácil de recorrer.
- b. Permite búsqueda binaria.*
- c. Permite búsqueda binaria sólo si las altas mantienen el archivo ordenado.*
- d. No permite búsqueda binaria si hay bajas lógicas.
- e. Algunas de las anteriores.*
- f. Ninguna de las anteriores.

81. Un archivo fragmentado:

- a. Debe compactarse para optimizar el espacio utilizado.
- b. No debe compactarse para optimizar el espacio utilizado.
- c. A veces, puede compactarse.*
- d. Nunca debe compactarse.
- e. Algunas de las anteriores.

82. Un archivo fragmentado:

- a. Debe compactarse para optimizar el espacio utilizado.
- b. No debe compactarse para optimizar el espacio utilizado.
- c. A veces, no puede compactarse.
- d. Nunca debe compactarse.
- e. Ninguna de las anteriores.*

83. Un archivo ordenado:

- a. Puede desordenarse.*
- b. Conviene mantenerlo ordenado.
- c. No conviene mantenerlo ordenado.
- d. No puede desordenarse.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

84. Un archivo organizado con registros de longitud variable:

- a. No permite realizar bajas lógicas.
- b. Optimiza la utilización de espacio en disco.*
- c. No permite realizar bajas físicas.
- d. Sólo acepta altas al final del archivo.

85. Un archivo que maneja registros de longitud fija necesita:

- a. Delimitadores que indiquen el fin de cada campo.
- b. Delimitadores que indiquen el fin de cada registro.
- c. Indicadores de longitud de registros.
- d. Ninguna de las anteriores.*

86. Un archivo secuencial:

- a. Debe contener registros de longitud fija.
- b. Debe contener registros de longitud variable.
- c. Puede contener registros de longitud fija.
- d. Puede contener registros de longitud variable.
- e. Permite que a y b sean correctas.
- f. Permite que a y c sean correctas.
- g. Permite que c y d sean correctas.**
- h. Permite que b y d sean correctas.

87. Un archivo serie:

- a. Debe ser de longitud fija.
- b. Debe ser de longitud variable.
- c. Puede ser de longitud fija.
- d. Puede ser de longitud variable.
- e. a y c son correctas.
- f. b y c son correctas.
- g. a y d son correctas.
- h. c y d son correctas.**

88. Un archivo serie:

- a. Está ordenado.
- b. Puede ordenarse.**
- c. Requiere ordenarse.
- d. No requiere ordenarse.
- e. No está ordenado.
- f. Algunas de las anteriores.

89. Un borrado lógico en un archivo de datos:

- a. Recupera inmediatamente el espacio borrado, dejando el archivo de tamaño menor.
- b. No se puede aplicar con registros de longitud variable.
- c. Sólo se aplica con registros de longitud fija.
- d. Permite recuperar el espacio con nuevas altas.**

90. Un índice primario:

- a. Se puede generar a partir de una clave unívoca de un registro.
- b. Se puede generar a partir de una clave no unívoca de un registro.
- c. Puede implementarse con una estructura de datos no lineal/lineal.**
- d. Debe implementarse con una estructura eficiente no lineal.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

91. Un índice secundario:

- a. Relaciona una clave secundaria con una o más claves primarias.**
- b. Puede repetir las claves.
- c. Puede organizarse con un árbol B*.**
- d. Ninguna de las anteriores.

92. Un índice secundario:

- a. Se puede implementar con un árbol binario de búsqueda.**
- b. Se puede implementar con una lista invertida.**
- c. Se puede implementar con un vector en memoria.
- d. Se puede implementar con una técnica de hashing.
- e. Todas las anteriores.
- f. Algunas de las anteriores.**

93. Un índice secundario asociado a un archivo:

- a. *Siempre debe referenciar al índice primario.*
- b. Siempre debe permitir acceder a los registros del archivo.
- c. A veces, debe permitir acceder a los registros del archivo.
- d. Puede referenciar al índice primario.
- e. Ninguna de las anteriores.

94. Una clave candidata:

- a. Admite repeticiones de valores.
- b. Admite repeticiones de campos.
- c. *Podría haber sido elegida como clave primaria.*
- d. Tiene exactamente las mismas características que la clave primarias.

95. Una clave permite:

- a. *Identificar un elemento particular dentro de un archivo.*
- b. Reconocer un conjunto de elementos con igual valor.
- c. *Ordenar lógicamente al archivo por los atributos que la componen.*
- d. Todas las anteriores.
- e. *Algunas de las anteriores.*
- f. Ninguna de las anteriores.

96. Una clave unívoca permite:

- a. *Identificar un elemento particular dentro de un archivo.*
- b. Reconocer un conjunto de elementos con igual valor.
- c. *Ordenar lógicamente al archivo por los atributos que la componen.*
- d. Todas las anteriores.
- e. *Algunas de las anteriores.*
- f. Ninguna de las anteriores.

97. Una operación de lectura en un archivo:

- a. *Mueve automáticamente al siguiente registro físicamente ordenado.*
- b. No mueve el puntero.
- c. Sólo mueve si no está en el último registro.
- d. *Mueve automáticamente el puntero.*
- e. Sólo se mueve el puntero después de un reset.
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

Preguntas Multiple Choice: **Fundamentos de Organización de Datos (Árboles).**

Los **árboles B** son árboles multicamino con una construcción especial que permite mantenerlos balanceados a bajo costo.

Un árbol B de orden M posee las siguientes propiedades básicas:

1. Cada nodo del árbol puede contener, como máximo, M descendientes y $M-1$ elementos.
2. La raíz no posee descendientes directos o tiene al menos dos.
3. Un nodo con x descendientes directos contiene $x-1$ elementos.
4. Los nodos terminales (hojas) tienen, como mínimo, $\lceil M/2 \rceil - 1$ elementos, y como máximo, $M-1$ elementos.
5. Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil M/2 \rceil$ elementos.
6. Todos los nodos terminales se encuentran al mismo nivel.

Un **árbol B+** es un árbol multicamino con las siguientes propiedades:

- Cada nodo del árbol puede contener, como máximo, M descendientes y $M-1$ elementos.
- La raíz no posee descendientes o tiene al menos dos.
- Un nodo con x descendientes contiene $x-1$ elementos.
- Los nodos terminales tienen, como mínimo, $(\lceil M/2 \rceil - 1)$ elementos, y como máximo, $M-1$ elementos.
- Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil M/2 \rceil$ descendientes.
- Todos los nodos terminales se encuentran al mismo nivel.
- Los nodos terminales representan un conjunto de datos y son enlazados entre ellos.

Un **árbol B*** es un árbol balanceado con las siguientes propiedades especiales:

- Cada nodo del árbol puede contener, como máximo, M descendientes y $M-1$ elementos.
- La raíz no posee descendientes o tiene al menos dos.
- Un nodo con x descendientes contiene $x-1$ elementos.
- Los nodos terminales tienen, como mínimo, $\lceil (2M-1)/3 \rceil - 1$ elementos, y como máximo, $M-1$ elementos.
- Los nodos que no son terminales ni raíz tienen, como mínimo, $\lceil (2M-1)/3 \rceil$ descendientes.
- Todos los nodos terminales se encuentran al mismo nivel.

1. Al trabajar con un árbol B:

- a. Cuando sucede overflow, algunas veces, se debe realizar el proceso de división del nodo.
- b. Cuando sucede underflow, algunas veces, se debe realizar el proceso de concatenación del nodo.*
- c. Cuando sucede overflow, algunas veces, se debe realizar el proceso de redistribución del nodo.
- d. Cuando sucede underflow, algunas veces, se debe realizar el proceso de redistribución del nodo.*
- e. Ninguna de las anteriores.

2. ¿Cuáles de las siguientes propiedades NO corresponde a un árbol B+ de orden M?:

- a. Cada nodo del árbol puede contener como máximo M descendientes y M-1 elementos.
- b. La raíz no posee descendientes o tiene al menos dos.
- c. Un nodo con x descendientes tiene x-1 elementos.
- d. Los nodos terminales tienen, como máximo, M-1 elementos.
- e. Los nodos no terminales pueden contener, como mínimo, $\lceil 2M/3 \rceil$ descendientes.
- f. Los nodos no terminales pueden contener, como mínimo, $\lceil M/2 \rceil$ descendientes.
- g. *Todas las anteriores.*

3. ¿Cuáles de las siguientes propiedades NO corresponde a un árbol B+ de orden M de prefijos simples?:

- a. Cada nodo del árbol puede contener como máximo M descendientes y M-1 elementos.
- b. La raíz no posee descendientes o tiene al menos dos.
- c. Un nodo con x descendientes tiene x-1 elementos.
- d. Los nodos terminales tienen, como máximo, M-1 elementos.
- e. Los nodos no terminales pueden contener, como mínimo, $\lceil 2M/3 \rceil$ descendientes.
- f. Los nodos no terminales pueden contener, como mínimo, $\lceil M/2 \rceil$ descendientes.
- g. Los elementos de datos se encuentran todos en nodos terminales.
- h. *Todas las anteriores.*

4. ¿Cuáles de las siguientes propiedades NO corresponde a un árbol B* de orden M?:

- a. Cada nodo del árbol puede contener máximo M descendientes y M-1 elementos.
- b. *La raíz no posee descendientes o posee $\lceil M/2 \rceil$ descendientes.*
- c. Todos los nodos terminales se encuentran al mismo nivel.
- d. Los nodos no terminales ni raíz tienen más de $\lceil M/2 \rceil$ y, a lo sumo, M descendientes.
- e. Un nodo con x descendientes contiene x-1 elementos.

5. ¿Cuáles de las siguientes definiciones pueden atribuirse a un árbol binario?:

- a. *Es una estructura de datos no lineal, en la cual cada nodo puede tener, a lo sumo, dos hijos.*
- b. Es una estructura de datos no lineal, que siempre se encuentra balanceada.
- c. Es una estructura de datos no lineal, que se encuentra balanceada en altura.
- d. Es una estructura de datos no lineal, en la cual cada nodo puede tener un número de hijos ilimitado.

6. ¿Cuáles de las siguientes definiciones pueden atribuirse a un árbol binario?:

- a. Es una estructura de datos lineal, en la cual cada nodo puede tener, a lo sumo, dos hijos.
- b. Es una estructura de datos no lineal, que siempre se encuentra balanceada.
- c. Es una estructura de datos no lineal, que se encuentra balanceada en altura.
- d. Es una estructura de datos no lineal, en la cual cada nodo puede tener un número de hijos ilimitado.
- e. *Ninguna de las anteriores.*

7. ¿Cuáles propiedades corresponden a un árbol B?:

- a. Cada nodo puede tener, como máximo, M descendientes, siendo M el orden del árbol.
- b. Un nodo que tiene x descendientes debe tener x-1 claves.
- c. Está siempre balanceado, sin importar los elementos que se inserten.
- d. *Todas las anteriores.*

8. ¿Cuáles propiedades corresponden a un árbol B* de orden M?:

- a. La diferencia máxima de altura entre los dos subárboles cualesquiera que compartan raíz es 1.
- b. Un nodo terminal tiene, como mínimo, $\lceil M/2 \rceil - 1$ claves.
- c. *Cada nodo puede tener, como máximo, M hijos.*
- d. *Un nodo no terminal que tiene K descendientes debe tener K-1 claves.*

e. Ninguna de las anteriores.

9. ¿Cuáles propiedades corresponden a un árbol B+ de prefijos simples?:

- a. Cada nodo puede tener, como máximo, M descendientes, siendo M el orden del árbol.
- b. Un nodo que tiene x descendientes debe tener x-1 claves.
- c. Está siempre balanceado, sin importar los elementos que se inserten.
- d. *Todas las anteriores.*

10. Con respecto a la paginación de un árbol binario:

- a. Cada página debe contener como mínimo 16 claves.
- b. Divide el árbol binario en páginas que almacena en memoria principal.
- c. *Para que sea más eficiente, es necesario que las páginas se ubiquen en direcciones cercanas.*
- d. Ninguna de las anteriores.

11. Con respecto a un árbol B*:

- a. Es más eficiente realizar una búsqueda sobre un árbol B que sobre un árbol B*.
- b. *La altura puede ser inferior a la de un árbol B porque los elementos se distribuyen más eficientemente en los nodos.*
- c. Permite acceder secuencialmente a los elementos.
- d. Ninguna de las anteriores.

12. Cuando un árbol tiende a llenarse:

- a. Se debe procurar más espacio para el archivo que lo contiene, reacomodando todos los nodos.
- b. Se debe procurar más espacio para el archivo que lo contiene, reacomodando el nodo padre y sus hermanos.
- c. Se debe procurar más espacio para el archivo.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

13. Cuando un árbol B tiende a llenarse:

- a. Se debe procurar más espacio para el archivo que lo contiene, reacomodando todos los nodos.
- b. Se debe procurar más espacio para el archivo que lo contiene, reacomodando el nodo padre y sus hermanos.
- c. Se debe procurar más espacio para el archivo.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

14. Cuando un árbol B+ de prefijos simples tiende a llenarse:

- a. Se debe procurar más espacio para el archivo que lo contiene, reacomodando todos los nodos.
- b. Se debe procurar más espacio para el archivo que lo contiene, reacomodando el nodo padre y sus hermanos.
- c. Se debe procurar más espacio para el archivo.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

15. Cuando se borra un elemento de un nodo en un árbol B:

- a. El elemento debe estar en un nodo terminal, si no lo está debe ser llevado a un nodo terminal.

- b. A veces, puede producirse underflow en el nodo y que esto produzca a una redistribución.
- c. A veces, puede producirse underflow en un nodo y que esto produzca una concatenación.
- d. *Todas las anteriores.*

16. Cuando se borra un elemento de un nodo en un árbol B:

- a. *El elemento debe estar en un nodo terminal, si no lo está debe ser llevado a un nodo terminal.*
- b. *A veces, puede producirse underflow en el nodo y que esto produzca a una redistribución.*
- c. Puede borrarse un elemento que no esté necesariamente ubicado en un nodo terminal.
- d. *Algunas de las anteriores.*

17. Cuando se inserta un elemento en un árbol binario:

- a. *Siempre se debe generar un nuevo nodo.*
- b. *Siempre es necesario acceder al nivel hoja.*
- c. Algunas veces, puede llegar a reducir la altura del árbol.
- d. Siempre aumenta la altura del árbol.
- e. Nunca aumenta la altura del árbol.
- f. *Algunas de las anteriores.*

18. Cuando se inserta un elemento en un árbol binario:

- a. Siempre se crea un nuevo nodo.
- b. Siempre es necesario acceder al nivel hoja.
- c. Puede siempre insertarse a la derecha del padre.
- d. Puede aumentar la altura del árbol.
- e. *Todas las anteriores.*
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

19. Cuando se realiza un alta en un árbol B:

- a. Se puede realizar en un nodo interno.
- b. Siempre se produce overflow.
- c. Puede llegar a necesitar realizar una fusión de nodos.
- d. *Siempre se llega hasta el nivel hoja.*
- e. Ninguna de las anteriores.

20. Cuando se realizan bajas en un árbol B:

- a. Siempre se aplica redistribución.
- b. Siempre se aplica fusión.
- c. *Siempre se accede a nivel hoja.*
- d. La altura del árbol siempre se reduce.
- e. *Algunas veces, puede llegar a reducir la altura del árbol.*
- f. *Algunas de las anteriores.*

21. Dado un árbol B de orden 100:

- a. *El nodo raíz puede tener sólo 3 hijos en algún momento de su construcción.*
- b. *El proceso de borrar un elemento del nodo raíz puede producir un underflow en un nodo terminal.*
- c. Puede ser que un nodo terminal con 50 elementos tenga 51 hijos.
- d. Todas las anteriores.
- e. *a y b son correctas.*
- f. b y c son correctas.
- g. a y c son correctas.
- h. Ninguna de las anteriores.

22. Dado un árbol B de prefijos simples de orden 100:

- a. Un nodo entra en underflow si se borra un elemento y solo quedan 48.
- b. Un nodo no terminal ni raíz tendrá, al menos, $\lceil M/2 \rceil$ hijos.
- c. Puede ser que la raíz tenga sólo dos hijos en algún momento.
- d. *Todas las anteriores.*
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

23. Dado un árbol B. Suponga que se tienen tres nodos terminales A, B y C. A adyacente hermano izquierda de B, C adyacente hermano derecha de B. Suponga que el nodo A está completo y que el nodo C tiene lugar disponible. Se inserta un elemento en el nodo B:

- a. Debe redistribuir con el nodo C si la política que se usa es la de derecha o izquierda.
- b. Debe dividirse con el nodo A si la política que se usa es la de izquierda.
- c. Debe redistribuir con el nodo C si la política que se usa es la de derecha.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

24. Dado un árbol B+:

- a. Todos sus nodos siempre tienen claves del archivo.
- b. Se lo puede utilizar sólo para recorrer secuencialmente al archivo.
- c. Es más eficiente que un árbol B en la búsqueda de un elemento.
- d. *Ninguna de las anteriores.*

25. Dado un árbol B+:

- a. Todos sus nodos siempre tienen el registro completo del archivo.
- b. Se lo puede utilizar sólo para recorrer secuencialmente al archivo.
- c. *Puede ser más ineficiente que un árbol B en la búsqueda de un elemento.*
- d. Ninguna de las anteriores.

26. Dado un árbol B* de orden 100:

- a. *El nodo raíz puede tener sólo 3 hijos en algún momento de su construcción.*
- b. Un nodo terminal puede producir un underflow si, al borrar un elemento, quedan 48 en el nodo.
- c. Puede ser que un nodo terminal con 50 elementos tenga 51 hijos.
- d. Todas las anteriores.
- e. a y b son correctas.
- f. b y c son correctas.
- g. a y c son correctas.
- h. Ninguna de las anteriores.

27. Dado un archivo con registros de longitud fija. Suponga que tiene un índice correspondiente a una clave unívoca. Suponga que se crea un árbol de orden 200 para

almacenar ese índice unívoco. Si el archivo y el árbol generado tuvieran 300 elementos insertados, entonces:

- a. Si se creara un árbol B, tendría la misma altura que crear un árbol B*.*
- b. Si se creara un árbol B, tendría mayor altura que un árbol B*.
- c. Si se creara un árbol B, tendría menor altura que un árbol B*.
- d. Ninguna de las anteriores.

28. Dado un archivo con registros de longitud fija. Suponga que tiene un índice correspondiente a una clave unívoca. Suponga que se crea un árbol de orden 200 para almacenar ese índice unívoca. Si archivo y el árbol generado tuvieran 300 elementos insertados, entonces:

- a. Un árbol B tiene la misma cantidad de nodos que un árbol B* creado para la misma finalidad.
- b. Un árbol B tiene más cantidad de nodos que un árbol B* creado para la misma finalidad.
- c. Un árbol B tiene menos cantidad de nodos que un árbol B* creado para la misma finalidad.
- d. Ninguna de las anteriores.*

29. En árboles B, la política izquierda o derecha, determina lo siguiente:

- a. Se intenta redistribuir con el hermano adyacente izquierdo, si no es posible, se intenta con el hermano adyacente derecho, si tampoco es posible, se fusiona con hermano adyacente izquierdo.*
- b. Se intenta redistribuir con el hermano adyacente derecho, si no es posible, se intenta con el hermano adyacente izquierdo, si tampoco es posible, se fusiona con hermano adyacente derecho.
- c. Se intenta fusionar con el hermano adyacente derecho, si no es posible, se intenta con el hermano adyacente izquierdo, si tampoco es posible, se redistribuye con hermano adyacente derecho.
- d. Se intenta fusionar con el hermano adyacente izquierdo, si no es posible, se intenta con el hermano adyacente derecho, si tampoco es posible, se redistribuye con hermano adyacente izquierdo.

30. En un árbol B:

- a. Cada nodo contiene X elementos y X-1 hijos.
- b. En algún caso, la raíz puede tener un solo hijo.
- c. Los nodos que contienen x elementos contienen x+1 hijos.
- d. Los nodos hojas pueden no estar al mismo nivel.
- e. Ninguna de las anteriores.*
- f. Algunas de las anteriores.

31. En un árbol B+:

- a. Para buscar un elemento, siempre se llega al nivel hoja.*
- b. Los nodos hojas no deben estar enlazados entre sí.
- c. Los nodos internos conforman un índice para llegar a un elemento buscado.*
- d. Algunas de las anteriores.*
- e. Ninguna de las anteriores.

32. En un árbol B de orden 50, cuando quedan 25 elementos en un nodo:

- a. Se produce underflow y, necesariamente, debe concatenarse con un adyacente hermano.
- b. Se produce underflow y, necesariamente, debe redistribuirse con un adyacente hermano.
- c. Se produce underflow y la operación a realizar depende del estado de los nodos adyacentes hermanos.
- d. No se produce underflow.*

33. La eficiencia de búsqueda de un árbol B:

- a. Es de orden lineal.
- b. Puede ser de orden lineal, bajo alguna circunstancia del árbol generado.
- c. Es de orden logarítmico.**
- d. Es de orden constante (orden del árbol).
- e. b y c son correctas.
- f. b, c y d son correctas.
- g. Ninguna de las anteriores.

34. La eficiencia de búsqueda de una clave en un árbol B+ es:

- a. De orden lineal.
- b. De orden logarítmico, similar a un árbol B.**
- c. De orden logarítmico, similar a un árbol B*.**
- d. De orden fijo, dado que los elementos de los nodos terminales están linkeados juntos.
- e. a, b y c son correctas.
- f. b, c y d son correctas.
- g. b y c son correctas.
- h. Ninguna de las anteriores.

35. La eficiencia promedio de búsqueda en un árbol B tiene:

- a. Orden lineal.
- b. Orden logarítmico.**
- c. Orden constante.
- d. Ninguna de las anteriores.

36. La eficiencia promedio de búsqueda en un archivo a partir de disponer de un índice implementado con un árbol del tipo B (B, B* o B+):

- a. Orden lineal.
- b. Orden logarítmico.**
- c. 1.
- d. No dispongo datos para contestar la pregunta.

37. Los árboles B*:

- a. Permiten localizar un registro de manera más eficiente que un árbol B, porque, además, permiten una búsqueda secuencial eficiente.
- b. Permiten localizar un registro de manera más eficiente que un árbol B, cuando ambos árboles tienen un solo nodo respectivamente.
- c. Completan los nodos en, al menos, 2/3 de su capacidad.**
- d. Los nodos terminales no aparecen en igual nivel.

38. Se definen 4 índices para un archivo de datos. Dichos índices se implementan con árboles balanceados:

- a. La eficiencia de los 4 árboles es de orden similar, es decir, orden lineal.
- b. Los 4 árboles tienen una eficiencia en términos matemáticos igual.
- c. *Alguno de los árboles puede ser más eficiente que otro en términos matemáticos.*
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

39. Se tiene un árbol B+ de orden M. Suponga que se haya utilizado una de las claves como separador. Si la clave se borra:

- a. Se debe borrar el separador.
- b. Se puede borrar el separador.
- c. *No se toca el separador.*
- d. Algunas de las anteriores.
- e. Ninguna de las anteriores.

40. Sea un árbol B de orden 100, el nodo X tiene 49 elementos, si se borra un elemento de dicho nodo:

- a. Sólo se borra el elemento.
- b. Se produce underflow y se debe concatenar el nodo x.
- c. Se produce underflow y se debe redistribuir el nodo x
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

41. Sea un árbol B+ de orden M:

- a. *Ante una inserción se puede producir división.*
- b. Ante una inserción con overflow siempre se produce división.
- c. Ante una inserción con overflow siempre se produce redistribución.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

42. Sea un archivo de alumnos, que maneja registros de longitud fija y que utiliza la técnica de primer ajuste para recuperar espacio en el caso de borrado. Si se implementa un índice por clave unívoca legajo:

- a. Se puede generar un árbol B para implantar ese índice.
- b. Se puede generar un árbol B* para implantar ese índice.
- c. Se puede generar un árbol B+ para implantar ese índice.
- d. Se puede implantar un árbol B+ de prefijos simples para implantar ese índice.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. *Ninguna de las anteriores.*

43. Sea un problema donde un índice correspondiente a una clave unívoca se implementa como un árbol balanceado:

- a. El archivo de datos debe ser de registros de longitud fija.
- b. *El archivo de datos puede ser de registros de longitud variable.*
- c. El archivo de datos no puede admitir bajas con recuperación de espacio.
- d. a y b son correctas.
- e. a y c son correctas.
- f. b y c son correctas.
- g. Ninguna de las anteriores.

44. Sea un problema donde un índice correspondiente a una clave unívoca se implementa como un árbol B*:

- a. El archivo de datos debe ser de registros de longitud fija.
- b. El archivo de datos debe ser de registros de longitud variable.
- c. *El archivo de datos puede admitir bajas con recuperación de espacio.*
- d. a y b son correctas.
- e. a y c son correctas.
- f. b y c son correctas.
- g. Ninguna de las anteriores.

45. Si el orden de un árbol B es 100 y, al borrar un elemento, quedan 48 en ese nodo:

- a. Se produce underflow y, necesariamente, debe concatenarse con un adyacente hermano.
- b. Se produce underflow y, necesariamente, debe redistribuirse con un adyacente hermano.
- c. *Se produce underflow y la operación a realizar depende del estado de los nodos adyacentes hermanos.*
- d. No se produce underflow.

46. Si el orden de un árbol B es 100 y, al borrar un elemento, quedan 49 en ese nodo:

- a. Se produce underflow y, necesariamente, debe concatenarse con un adyacente hermano
- b. Se produce underflow y, necesariamente, debe redistribuirse con un adyacente hermano
- c. Se produce underflow y la operación a realizar depende del estado de los nodos adyacentes hermanos.
- d. *No se produce underflow.*

47. Si se implementa un algoritmo que permita generar un árbol B+, ese algoritmo:

- a. No necesita manipular el puntero al nodo raíz.
- b. No necesita manipular el puntero al nodo de los elementos de datos menores.
- c. *La estructura interna del árbol puede implementarse como B*.*
- d. Todas las anteriores.
- e. Algunas de las anteriores.

48. Suponga que el nodo terminal de un árbol B+ de prefijos simples de orden 7 tiene las claves enteras siguientes 10342, 10563, 11763, 12345, 13423, 13443 y se inserta en ese nodo la clave 15000:

- a. Se produce overflow y, luego de dividir, al padre del nodo se sube la clave 12.
- b. Se produce overflow y, luego de dividir, al padre del nodo se sube la clave 11.
- c. *Se produce overflow y, luego de dividir, se sube la clave 12345.*
- d. Todas las anteriores pueden ser correctas, depende el algoritmo utilizado.
- e. a y b pueden ser correctas, dependen del algoritmo utilizado.
- f. a y c pueden ser correctas, dependen del algoritmo utilizado.
- g. b y c pueden ser correctas, dependen del algoritmo utilizado.
- h. Ninguna de las anteriores.

49. Suponga que el nodo terminal de un árbol B+ de prefijos simples de orden 7 tiene las claves GONZALEZ, GOÑEZ, GOODMAN, GOPLANI, GORBA y, en dicho nodo, se inserta una clave nueva GUTIERREZ, entonces:

- a. Se produce overflow y, luego de dividir, al padre del nodo se sube la clave GOP.
- b. Se produce overflow y, luego de dividir, al padre del nodo se sube la clave GOO.
- c. Se produce overflow y, luego de dividir, se sube la clave GO.
- d. Todas las anteriores pueden ser correctas, depende el algoritmo utilizado.
- e. a y b pueden ser correctas, dependen del algoritmo utilizado.
- f. a y c pueden ser correctas, dependen del algoritmo utilizado.
- g. b y c pueden ser correctas, dependen del algoritmo utilizado.
- h. *Ninguna de las anteriores.*

50. Suponga que se genera un árbol binario para implantar un índice de un archivo. El índice es por la clave unívoca legajo que ocupa 10 bytes. Entonces, cada nodo del árbol ocupará:

- a. 18 bytes.
- b. A lo sumo, 18 bytes.
- c. Más de 21 bytes.**
- d. 10 bytes.
- e. Ninguna de las anteriores.

51. Suponga que sobre un nodo de un árbol B* se produce overflow. En dicho caso, se puede:

- a. Aplicar saturación progresiva encadenada.
- b. Aplicar doble dispersión.
- c. Aplicar un área de desborde separada para el nodo.
- d. Redistribución.**
- e. División.**
- f. Todas las anteriores.
- g. Algunas de las anteriores.**
- h. Ninguna de las anteriores.

52. Un árbol AVL:

- a. *Es binario.*
- b. Está balanceado.
- c. No puede estar completamente balanceado.
- d. Si es binario, está balanceado.
- e. Si no es binario, puede no estar balanceado completamente.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

53. Un árbol AVL:

- a. *Tiene eficiencia de búsqueda logarítmica.*
- b. Puede tener eficiencia de búsqueda logarítmica.
- c. puede tener eficiencia de búsqueda lineal.
- d. a y b son correctas.
- e. b y c son correctas.
- f. a y c son correctas.
- g. Ninguna de las anteriores.

54. Un árbol AVL es:

- a. Un árbol n-ario ($n > 2$).
- b. Un árbol B.
- c. Un árbol binario paginado.
- d. *Un árbol binario balanceado en altura (BA(1)).*
- e. Ninguna de las anteriores.

55. Un árbol balanceado de orden 200:

- a. Se desbalancea cuando la raíz es el único nodo del árbol y ésta produce overflow en una inserción.
- b. Puede contener un nodo con 99 hijos y 99 claves.
- c. Dos nodos adyacentes hermanos, de diferente padre, pueden estar completos, es decir, con 199 elementos.
- d. Todos los nodos terminales están a la misma distancia de todos los nodos raíz.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. *Ninguna de las anteriores.*

56. Un árbol B:

- a. Puede ser un árbol AVL.
- b. *Puede guardarse en memoria RAM.*
- c. *Puede implementar una clave no unívoca.*
- d. *Puede tener acceso secuencial eficiente y rápido.*
- e. Todas las anteriores.
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

57. Un árbol B de orden 130:

- a. Puede tener la raíz con sólo dos hijos.
- b. Tiene todos los nodos terminales en igual nivel.
- c. Puede tener todos sus nodos ocupados en, al menos, 2/3 de su capacidad.
- d. *Todas las anteriores.*
- e. a y b son correctas.
- f. b y c son correctas.
- g. a y c son correctas.
- h. Ninguna de las anteriores.

58. Un árbol B de orden 200:

- a. En una hoja, puede tener hasta 99 elementos.
- b. En una hoja, puede tener más de 99 elementos.**
- c. En una hoja, puede tener menos de 99 elementos.
- d. La raíz siempre tiene hijos.
- e. La raíz tiene hijos si el árbol tiene más de 50 elementos.

59. Un árbol B es más eficiente que un árbol B+:

- a. Porque tiene un algoritmo de inserción más eficiente.
- b. Porque tiene un algoritmo de borrado más eficiente.
- c. Porque tiene un algoritmo de búsqueda más eficiente.
- d. No, un árbol B no es más eficiente que un árbol B+.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.**

60. Un árbol B+:

- a. Al realizar una inserción, puede tener nodos hojas con underflow.
- b. Al realizar una baja, puede tener nodos hojas con overflow.
- c. Al realizar una inserción, puede requerirse concatenación.
- d. Al realizarse una baja, puede requerirse división.
- e. Al realizarse un alta, puede requerirse redistribución.
- f. Todas las anteriores.
- g. Algunas de las anteriores.
- h. Ninguna de las anteriores.**

61. Un árbol B+:

- a. Puede ser de prefijos simples.
- b. Si es de prefijos simples, está ordenado.
- c. Si no es de prefijos simples, está ordenado.
- d. Está siempre ordenado.
- e. Todas las anteriores.**
- f. Algunas de las anteriores.

62. Un árbol B+:

- a. Siempre tiene más claves que un árbol B, para el mismo archivo de datos.
- b. Siempre tiene más claves que un árbol B*, para el mismo archivo de datos.
- c. Siempre es más alto que un árbol B.
- d. Siempre es más alto que un árbol B*.
- e. Todas de las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.**

63. Un árbol B+ de orden M:

- a. Es un árbol multicamino.**
- b. Es un árbol balanceado.**
- c. Es un árbol que en cada nodo (salvo la raíz) tiende a llenarse en al menos $2/3$.
- d. Algunas de las anteriores.**
- e. Ninguna de las anteriores.

64. Un árbol B+ de prefijos simples:

- a. A veces, no tiene prefijos.
- b. Puede no tener prefijos simples.
- c. Si tiene prefijos simples, las hojas están enlazadas.
- d. Si no tiene prefijos simples, las hojas están enlazadas.

e. Todas las anteriores.

f. Algunas de las anteriores.

g. Ninguna de las anteriores.

65. Un árbol B+ de prefijos simples:

a. Se utiliza para ordenar físicamente un archivo.

b. Se utiliza para lograr acceso rápido a la información de un archivo.

c. Se utiliza para lograr acceso secuencial rápido a un archivo.

d. Se utiliza para lograr acceso directo a los elementos de un archivo.

e. Ninguna de las anteriores.

66. Un árbol B+ de prefijos simples:

a. Se utiliza para representar un índice de un archivo.

b. Se utiliza para lograr acceso secuencial rápido a un archivo.

c. Optimiza el espacio requerido para almacenar el árbol.

d. Todas las anteriores.

67. Un árbol B+ de prefijos simples de orden M:

a. Se puede aplicar a una clave única, cuyo atributo es un string.

b. Se puede aplicar a una clave secundaria, cuyo atributo es un string.

c. Se puede aplicar a una clave primaria, cuyo atributo es un entero.

d. Se puede aplicar sólo a una clave unívoca, cuyo atributo es un string.

e. Todas las anteriores.

f. a y b son correctas.

g. a, c y d son correctas.

h. a, b y d son correctas.

i. Ninguna de las anteriores.

68. Un árbol B*:

a. Distribuye las claves de manera más eficiente que un árbol B.

b. La altura puede ser inferior a la de un árbol B+ porque los elementos se distribuyen más eficientemente en los nodos.

c. La altura puede ser superior a la de un árbol B porque los elementos se distribuyen más eficientemente en los nodos.

d. Permite acceder secuencialmente a los elementos del árbol.

e. Hay dos respuestas anteriores correctas.

69. Un árbol B*:

a. Es más eficiente en el algoritmo de búsqueda que un árbol B.

b. La altura puede ser inferior a la de un árbol B porque los elementos se distribuyen más eficientemente en los nodos.

c. La altura puede ser superior a la de un árbol B porque los elementos se distribuyen más eficientemente en los nodos.

d. Permite acceder secuencialmente a los elementos del árbol.

70. Un árbol B*:

a. Puede ser un árbol binario de búsqueda.

b. Puede implementarse con una estructura de datos lineal.

c. Puede tener elementos repetidos, o sea implantar un índice secundario.

d. Puede tener acceso secuencial eficiente y rápido.

e. Todas las anteriores.

f. Algunas de las anteriores.

g. Ninguna de las anteriores.

71. Un árbol B*:

a. Se construye de manera similar a un árbol binario, salvo que en todos los nodos cabe más de un elemento.

b. Se construye de manera similar a un árbol B.

c. Puede ser la estructura superior (separadores) de un árbol B+.

d. Algunas de las anteriores.

e. Ninguna de las anteriores.

72. Un árbol B*:

a. Todos los nodos menos la raíz seguro están llenos a 2/3 de su capacidad en todo momento.

b. Los nodos terminales puede, en situaciones especiales, tener menos de 2/3 de su capacidad ocupada.

c. El nodo adyacente hermano de uno que entra en overflow siempre se puede usar para redistribuir, si no estuviera completo.

d. Todas las anteriores.

e. a y b son correctas.

f. b y c son correctas.

g. a y c son correctas.

h. Ninguna de las anteriores.

73. Un árbol B* es más eficiente que un árbol B:

a. Porque tiene un algoritmo de inserción más eficiente.

b. Porque tiene un algoritmo de borrado más eficiente.

c. Porque tiene un algoritmo de búsqueda más eficiente.

d. No, un árbol B* no es más eficiente que un árbol B.

e. Todas las anteriores.

f. Algunas de las anteriores

g. Ninguna de las anteriores.

74. Un árbol binario:

a. Es una estructura de datos lineal, en la cual cada nodo puede tener, a lo sumo, dos hijos.

b. Es una estructura de datos no lineal, que siempre se encuentra balanceada.

c. Es una estructura de datos no lineal, donde cada nodo tiene dos hijos.

d. Es una estructura de datos lineal que se puede desbalancear.

e. Es una estructura de datos no lineal que puede llegar a tener un orden lineal de búsqueda.

75. Un árbol binario:

a. Permite implementar un índice secundario de un archivo.

b. Es la mejor solución para implementar un índice a partir de la clave primaria de un archivo.

c. Es la única solución para implementar el índice correspondiente a una clave unívoca.

d. Cada elemento del árbol tiene un hijo izquierda y un hijo derecha

e. Todas las anteriores.

f. Algunas de las anteriores.

g. Ninguna de las anteriores.

76. Un árbol binario:

a. Puede tener eficiencia de búsqueda logarítmica.

b. Puede estar balanceado si tiene 127 elementos.

c. Puede tener eficiencia de búsqueda lineal.

d. Todas las anteriores.

e. Algunas de las anteriores

f. Ninguna de las anteriores.

77. Un árbol binario:

a. Tiene igual eficiencia para la búsqueda de información que un árbol B*.

b. Tiene igual eficiencia para la búsqueda de información que un árbol B* con prefijos simples.

c. Se desbalancea fácilmente.

d. Ninguna de las anteriores.

78. Un árbol binario de orden 4:

a. Puede desbalancearse.

b. Puede balancearse.

c. Si se emplean los algoritmos correctos, puede quedar balanceado en altura.

d. Si está desbalanceado, no puede presentar una eficiencia de búsqueda de orden logarítmica.

e. Todas las anteriores.

f. Algunas de las anteriores.

g. Ninguna de las anteriores.

79. Un árbol que NO se encuentra balanceado:

a. Puede ser un árbol binario.

b. Puede ser un árbol multicamino.

c. Puede ser un árbol binario paginado.

d. No puede ser un árbol B+.

e. No puede ser un árbol B*.

f. Todas las anteriores.

g. Ninguna de las anteriores.

80. Un árbol que se encuentra balanceado:

a. Puede ser un árbol binario.

b. Puede ser un árbol multicamino.

c. Puede ser un árbol B+.

d. Puede ser un árbol B*.

e. Todas las anteriores.

f. c y d son correctas.

g. b, c y d son correctas.

h. b y d son correctas.

i. Ninguna de las anteriores.

81. Un árbol multicamino es:

a. Es una estructura de datos lineal, en la cual cada nodo puede tener un número indeterminado de hijos.

b. Es una estructura de datos no lineal, que siempre se encuentra balanceada.

c. Es una estructura de datos no lineal, que se encuentra balanceada en altura.

d. Ninguna de las anteriores.

82. Un árbol multicamino es:

a. Es una estructura de datos no lineal, en la cual cada nodo puede tener un número determinado de hijos.

b. Es una estructura de datos no lineal, que siempre se encuentra balanceada.

c. Es una estructura de datos no lineal, que se encuentra balanceada en altura.

d. Es una estructura de datos no lineal, en la cual cada nodo puede tener, a lo sumo, 5 hijos.

83. Un índice implementado con una lista invertida:

a. Debe proveer acceso rápido (eficiente) a un registro.

b. Puede proveer acceso rápido (eficiente) a un registro.

c. Debe proveer acceso secuencial rápido (eficiente) a todos los registros.

d. Puede proveer acceso secuencial rápido (eficiente) a todos los registros.

e. Todas las anteriores.

f. Algunas de las anteriores.

g. Ninguna de las anteriores.

84. Un índice primario es:

- a. Una estructura de datos adicional que contiene el mismo volumen de información que el archivo original.
- b. Una estructura de datos adicional que permite ordenar físicamente el archivo original.
- c. *Una estructura de datos adicional que agilizar el acceso a la información del archivo.*
- d. Una estructura de datos adicional que puede contener mayor volumen de información que el archivo original.
- e. Ninguna de las anteriores.

85. Un índice secundario es:

- a. *Una estructura de datos adicional que permite asociar una o varias claves primarias con una clave secundaria.*
- b. Una estructura de datos adicional que contiene el mismo volumen de información que el archivo original.
- c. Una estructura de datos adicional que ordena físicamente (en memoria secundaria) el archivo original.
- d. Una estructura de datos adicional que permite relacionar una clave secundaria con una sola clave primaria.
- e. Todas las anteriores.

86. Un índice secundario tiene eficiencia de búsqueda:

- a. Lineal.
- b. Logarítmica.
- c. Constante y es 1.
- d. Constante y puede tender a 1.
- e. Algunas de las anteriores.
- f. *Ninguna de las anteriores.*

87. Una estructura tipo árbol:

- a. Siempre tiene eficiencia de búsqueda logarítmica.
- b. Algunas veces, tiene eficiencia de búsqueda constante.
- c. Si es B+, algunas veces, tiene eficiencia de búsqueda logarítmica.
- d. Algunas de las anteriores.
- e. *Ninguna de las anteriores.*

88. Una inserción en un nodo cualquiera (terminal) de árbol B:

- a. Puede generar overflow.
- b. Puede generar división de un solo nodo.
- c. Pueden dividirse tres nodos.
- d. *Todas las anteriores.*
- e. a y b s son correctas.
- f. b y c son correctas.
- g. a y c son correctas.

Preguntas Multiple Choice: **Fundamentos de Organización de Datos (Hashing).**

1. A partir de un archivo dispersado con hashing extensible:

- a. *Siempre es posible agregar elementos al archivo.*
- b. Algunas veces, es posible agregar elementos al archivo.
- c. Se puede utilizar saturación progresiva / saturación progresiva encadenada / dispersión doble para tratar registros en saturación.
- d. *No se puede utilizar saturación progresiva / dispersión doble para tratar registros en saturación.*
- e. *Algunas de las anteriores.*
- f. Ninguna de las anteriores.

2. Al usar dispersión con espacio de direccionamiento estático, ¿cuáles de los siguientes procesos se debe realizar si se agota el espacio disponible asignado al archivo?

- a. Iniciar un nuevo archivo y relacionarlo con el archivo que quedó completo.
- b. *Obtener más espacio para el mismo archivo, actualizar la función de hash y redispersar el archivo completo.*
- c. Obtener más espacio para el mismo archivo, actualizar la función de hash pero no redispersar (se usa la nueva función sólo para los nuevos elementos).
- d. No es posible que se agote el espacio disponible asignado al archivo.
- e. Ninguna de las anteriores.

3. Al utilizar la técnica de dispersión doble:

- a. Dada una clave, siempre se debe aplicar dos funciones: inicialmente se aplica la 1ra función, y al resultado se le aplica la 2da función para obtener, finalmente, la dirección de almacenamiento.
- b. *Se cuenta con dos funciones, pero sólo se aplica la 2da función si se produjo una saturación al aplicar la 1ra función.*
- c. Cuando se usa la 2da función, el valor obtenido reemplaza al anteriormente obtenido por la 1ra función.
- d. Ninguna de las anteriores.

4. Con respecto a la dispersión extensible:

- a. El espacio aumenta o disminuye dependiendo de los registros que contiene el archivo.
- b. Necesita una tabla auxiliar.**
- c. La densidad de empaquetamiento siempre se mantiene por debajo del 75%.
- d. Ninguna de las anteriores.

5. ¿Cuál de las siguientes consignas NO define hash dinámico?:

- a. Recupera los registros en un acceso a disco.
- b. No puede haber estructuras adicionales.**
- c. Se organiza todo el archivo de datos.
- d. Sólo puede organizarse por un único criterio, la clave primaria.
- e. Todas las anteriores.

6. ¿Cuál de las siguientes definiciones corresponden al método de hash?:

- a. Técnica para generar una dirección base única para una clave dada.
- b. Técnica que convierte la clave asociada a un registro de datos en un número aleatorio, que se utiliza para determinar dónde se almacena el registro.
- c. Técnica de almacenamiento y recuperación que usa una función para mapear registros en direcciones de almacenamiento en memoria secundaria.
- d. Todas las anteriores.**

7. ¿Cuál de los siguientes conceptos corresponden con parámetros de la dispersión?:

- a. Capacidad de almacenamiento de cada sector del archivo.
- b. Densidad de empaquetamiento.
- c. Método de tratamiento de desbordes.
- d. Todos los anteriores.**

8. ¿Cuál de los siguientes métodos sirve para el tratamiento de colisiones en hash estático?:

- a. Área de desborde por separado.
- b. Saturación progresiva.
- c. Saturación progresiva encadenada
- d. Doble dispersión.
- e. Todas las anteriores.
- f. Ninguna de las anteriores.**

9. ¿Cuál de los siguientes parámetros de la técnica de hash es más importante?:

- a. Función de hash.
- b. Tamaño de cada nodo de almacenamiento.
- c. Densidad de empaquetamiento.
- d. Método de tratamiento de overflow.
- e. Ninguno es importante.
- f. Ninguno prevalece sobre el otro.**

10. ¿Cuáles de los siguientes parámetros NO corresponde a hashing?:

- a. Función de hash.
- b. Tamaño del nodo (capacidad para almacenar registros).
- c. Densidad de búsqueda.**
- d. Forma de tratar los desbordes.
- e. Todas las anteriores.

11. ¿Cuáles de estas técnicas se pueden utilizar con un archivo de registros de longitud fija?:

- a. Dispersión doble.**
- b. Hashing asistido por tabla.**
- c. Hashing extensible.**
- d. Ninguna de las anteriores.

12. ¿Cuáles de los siguientes parámetros afecta la eficiencia de la dispersión?:

- a. Cantidad de elementos del archivo.
- b. Cantidad de espacio para almacenar el archivo.
- c. Densidad de empaquetamiento.
- d. Función de dispersión.
- e. Algoritmos para el tratamiento de registros en saturación.
- f. Todas de las anteriores.**
- g. Algunas de las anteriores.
- h. Ninguna de las anteriores.

13. ¿Cuáles de los siguientes parámetros afecta la eficiencia de la dispersión?:

- a. Cantidad de elementos sinónimos del archivo.
- b. Cantidad de espacio para almacenar el archivo.**
- c. Densidad de empaquetamiento.**
- d. Función de dispersión.**
- e. Algoritmos para el tratamiento de registros en saturación.**
- f. Todas de las anteriores.
- g. Algunas de las anteriores.**
- h. Ninguna de las anteriores.

14. ¿Cuáles de los siguientes ítems pueden considerarse atributos o propiedades de la técnica de hash?:

- a. No requerir espacio adicional de almacenamiento.**
- b. Facilitar inserción y eliminación rápida de elementos en el archivo.**
- c. Permitir la búsqueda más eficiente de información en un archivo de datos.**
- d. Permitir acceso secuencial a los datos.
- e. Todas las anteriores.
- f. Algunas de las anteriores.**

15. Con 10.000 direcciones con capacidad para 4 registros cada una y 30.000 claves para dispersar, entonces:

- a. La densidad de empaquetamiento es mayor que uno.
- b. La densidad de empaquetamiento es superior o igual al 75%.**
- c. La densidad de empaquetamiento es inferior al 75% pero mayor o igual al 50%.
- d. La densidad de empaquetamiento es inferior al 50% pero mayor o igual al 25%.
- e. Ninguna de las anteriores.

16. Cuando la densidad de empaquetamiento de un archivo tiende a uno:

- a. Es necesario redefinir el espacio disponible únicamente.
- b. El archivo se completa y no es posible incorporar más elementos.
- c. Se debe cambiar la política de hash de estática a dinámica.
- d. Es necesario redefinir el espacio disponible y rehashear todo el archivo.**

17. Cuando se produce un overflow:

- a. Se puede aplicar el método de doble dispersión.
- b. Se puede aplicar el método de área separada.
- c. Se puede crear un nodo nuevo.
- d. Se puede aplicar el método de saturación progresiva.
- e. Todas las anteriores.**
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

18. Cuando una clave “x” y otra clave “y” generan, por función de dispersión, una misma dirección, entonces:

- a. Una de las dos no será almacenada en el archivo.
- b. Se produce un desborde.
- c. *Se produce una colisión.***
- d. Algunas de las anteriores.
- e. Ninguna de las anteriores.

19. Cuando una clave “x” y otra clave “y” generan, por función de dispersión, diferente dirección, entonces:

- a. Una de las dos no será almacenada en el archivo.
- b. *Se puede producir un desborde.***
- c. Se produce una colisión.
- d. Algunas de las anteriores.
- e. Ninguna de las anteriores.

20. El hash con espacio de direccionamiento dinámico:

- a. Encuentra los registros con un solo acceso a disco.*
- b. Se puede aplicar a cualquier clave.
- c. No requiere de espacio adicional.
- d. Todas la anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

21. El hash con espacio de direccionamiento dinámico:

- a. Utiliza una función de hash diferente cada vez que el tamaño del archivo crece.
- b. Aumenta la capacidad del nodo cuando se puede producir un overflow.
- c. Aumenta la cantidad de nodos en caso de overflow.*
- d. Todas la anteriores.
- e. Ninguna de las anteriores.

22. El hash con espacio de direccionamiento estático:

- a. Puede tener densidad de empaquetamiento menor que uno.
- b. Puede tener un tratamiento de desbordes.
- c. Puede tener una función aleatoria y uniforme.
- d. Todas las anteriores.
- e. a y b son correctas.
- f. a y c son correctas.
- g. b y c son correctas.
- h. Ninguna de las anteriores.*

23. El hashing extensible:

- a. Afecta a la densidad de empaquetamiento.
- b. Puede afectar a la densidad de empaquetamiento.
- c. Afecta la densidad de empaquetamiento sólo en casos especiales.
- d. Afecta la densidad de empaquetamiento en la mayoría de los casos.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.*

24. El método de área de desborde separada:

- a. Reubica los registros en overflow.*
- b. Utiliza una segunda función de hash en caso de ser necesaria.
- c. Puede generar áreas de overflow dentro del archivo.
- d. Todas las anteriores.
- e. Algunas de las anteriores
- f. Ninguna de las anteriores.

25. El método de área de desborde por separado:

- a. Utiliza una segunda función de hash para ubicar los registros en saturación de un archivo.
- b. Direcciona el overflow de un nodo a otro nodo diferente.
- c. Evita generar zonas contiguas de nodos en overflow.
- d. Todas las anteriores.
- e. a y b son correctas.
- f. a y c son correctas.
- g. b y c son correctas.*
- h. Ninguna de las anteriores.

26. El método de área de desborde por separado:

- a. Utiliza una segunda función de hash para ubicar los registros en saturación de un archivo.
- b. Ubica los registros lo más próximo posible a su dirección base.

c. Evita generar zonas contiguas de nodos en overflow.

- d. Todas las anteriores.
- e. a y b son correctas.
- f. a y c son correctas.
- g. b y c son correctas.
- h. Ninguna de las anteriores.

27. El método de dispersión doble:

- a. Afecta a la densidad de empaquetamiento.
- b. Puede afectar a la densidad de empaquetamiento.
- c. Afecta la densidad de empaquetamiento sólo en casos especiales.
- d. Afecta la densidad de empaquetamiento en la mayoría de los casos.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.*

28. El método de doble dispersión:

- a. Utiliza una segunda función de hash para ubicar a todos los registros del archivo.
- b. Utiliza una segunda función de hash para ubicar algunos registros del archivo.
- c. Evita generar zonas contiguas de nodos en overflow.
- d. Todas las anteriores.
- e. a y b son verdaderas.
- f. a y c son correctas.
- g. b y c son correctas.*
- h. Ninguna de las anteriores.

29. El método de doble dispersión para el tratamiento de colisiones:

- a. Utiliza una segunda función de hash para ubicar a todos los registros del archivo.
- b. Utiliza una segunda función de hash para ubicar algunos registros del archivo.
- c. Evita generar zonas contiguas de nodos en overflow.
- d. Todas las anteriores.
- e. a y b son verdaderas.
- f. a y c son correctas.
- g. b y c son correctas.
- h. Ninguna de las anteriores.*

30. El método de saturación progresiva encadenada:

- a. Es más eficiente que doble dispersión porque utiliza una sola función de hash.
- b. Se demora siempre menos en la búsqueda que utilizando el método de área separado.
- c. Es un método más de tratamiento de colisiones.
- d. Siempre es más eficiente que cualquier situación similar resultante con el método de saturación progresiva.*

31. El método de tratamiento de desbordes:

- a. Afecta la DE.
- b. Puede afectar la DE.
- c. Afecta la DE en casos especiales.
- d. Afecta la DE en la mayoría de los casos.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.*

32. El método de tratamiento de desbordes en área separada:

- a. Utiliza una segunda función de hash para determinar dónde va el registro en overflow.

- b. Utiliza una segunda función de hash para determinar el nuevo nodo donde se guardará el registro en overflow.
- c. Selecciona el primer nodo libre más cercano al nodo saturado.
- d. Selecciona el primer nodo libre más cercano al nodo saturado y los linkea.
- e. Ninguna de las anteriores.*

33. En un ambiente de dispersión, cuánto más grande es el tamaño de la cubeta:

- a. Hay más fragmentación.*
- b. Hay mayor probabilidad de saturación.
- c. La búsqueda dentro de la cubeta es más lenta.
- d. Ninguna de las anteriores.

34. En un ambiente de dispersión con espacio de direccionamiento estático:

- a. Siempre se encuentra un registro con un solo acceso a disco.
- b. No se permiten claves duplicadas.*
- c. Es posible usar archivos con registros de longitud variable.
- d. Ninguna de las anteriores.

35. En un ambiente de dispersión con espacio de almacenamiento estático:

- a. Se debe usar archivos con registro de longitud fija.*
- b. Siempre se encuentra un registro con un solo acceso a disco.
- c. No existe orden físico de datos.
- d. Ninguna de las anteriores.

36. Implementar hash para ordenar un archivo:

- a. Mejora el acceso a un archivo.
- b. Puede empeorar el acceso a los datos de un archivo.
- c. Si es hash extensible la mejora es notable.
- d. *El concepto no es aplicable.*

37. Indique cuál método de tratamiento de desborde potencialmente podría generar desplazamientos de disco (y, consecuentemente, demorar más tiempo) para encontrar un registro en saturación. Para este análisis, debe suponer que el registro en saturación se aloja en la primera dirección disponible de acuerdo a la política de cada método:

- a. Saturación progresiva.
- b. Saturación progresiva encadenada.
- c. Doble dispersión.
- d. *Área de desborde separada.*

38. La DE:

- a. Es un parámetro de eficiencia en cualquier tipo de hashing.
- b. Es un parámetro de eficiencia sólo para un tipo de hashing.**
- c. Permite detectar si la cantidad de espacio libre en el archivo puede crecer.
- d. Permite detectar si la cantidad de elementos del archivo puede crecer.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

39. La densidad de empaquetamiento en un archivo con registros con longitud variable:

- a. Se calcula como el cociente entre la cantidad de registros del archivo y la cantidad de espacio disponible.
- b. Es útil para establecer la proporción de espacio del archivo asignado que en realidad almacena registros.
- c. A medida que disminuye, aumenta la probabilidad de overflow.
- d. A medida que disminuye, hay más desperdicio de espacio.
- e. Alguna de las anteriores.
- f. Ninguna de las anteriores.**

40. La densidad de empaquetamiento se define como:

- a. El cociente entre cantidad de registros y espacio disponible en el archivo.**
- b. El cociente entre la cantidad de registros y la cantidad de nodos del archivo.
- c. El cociente entre la cantidad de registros, y el producto entre la cantidad de nodos y el contenido posible de registros de cada nodo.**
- d. Algunas de las anteriores.**

41. La densidad de empaquetamiento se puede definir como:

- a. $DE = \text{cantidad de registros} / (\text{cantidad de cubetas} * \text{capacidad de cubeta})$.**
- b. La proporción de espacio asignado al archivo que, en realidad, almacena registros.**
- c. La relación entre la cantidad de registros y la cantidad de cubetas del archivo.
- d. Ninguna de las anteriores.
- e. Algunas de las anteriores.**

42. La densidad de empaquetamiento se puede definir como:

- a. La relación entre la cantidad de registros y la cantidad de cubetas del archivo.
- b. La proporción de espacio asignado al archivo que, en realidad, almacena registros.**
- c. $DE = \text{cantidad de registros} / (\text{cantidad de cubetas})$.
- d. Ninguna de las anteriores.

43. La dispersión dinámica, denominada hash extensible:

- a. Siempre requiere el uso de una estructura auxiliar.**
- b. Guarda los registros de forma ordenada por algún criterio.
- c. Necesita de dos funciones de dispersión.
- d. Siempre requiere variar el tamaño del espacio de direcciones.
- e. Ninguna de las anteriores.

44. La dispersión dinámica, denominada hash extensible:

- a. Siempre requiere el uso de una estructura auxiliar.
- b. Guarda los registros de forma ordenada por algún criterio.
- c. Necesita de dos funciones de dispersión.
- d. Varía el tamaño del espacio de direcciones disponible, sin afectar a la función de hash.
- e. a y b son correctas.
- f. a y d son correctas.**
- g. Ninguna de las anteriores.

45. La eficiencia de búsqueda de un registro en un archivo organizado mediante hashing dinámico:

- a. Es de orden lineal.
- b. Tiende a uno.
- c. Siempre es uno.**
- d. Es de orden logarítmico.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

46. La eficiencia de búsqueda de un registro en un archivo organizado mediante hashing estático tiene:

- a. Orden lineal.
- b. Algunas veces, es uno.**
- c. Siempre es uno.
- d. Orden logarítmico.

47. La eficiencia de búsqueda de un registro en un archivo organizado mediante hashing estático:

- a. Es de orden lineal.
- b. Tiende a uno.**
- c. Siempre es uno.
- d. Es de orden logarítmico.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.

48. La eficiencia promedio de búsqueda en un archivo a partir de estar organizado mediante política de hashing:

- a. Orden lineal.
- b. Orden logarítmico.
- c. Orden constante.**
- d. Ninguna de las anteriores.

49. La función de hashing:

- a. Afecta a la densidad de empaquetamiento.
- b. Puede afectar a la densidad de empaquetamiento.
- c. Afecta la densidad de empaquetamiento sólo en casos especiales.
- d. Afecta la densidad de empaquetamiento en la mayoría de los casos.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.**

50. La técnica de dispersión de archivos se utiliza para:

- a. Mantener el archivo físicamente ordenado.
- b. Mejorar la performance de las inserciones.**
- c. Mejorar la performance de las bajas.**
- d. No realizar grandes desplazamientos en disco.
- e. Algunas de las anteriores.**
- f. Ninguna de las anteriores.

51. La técnica de área de desborde por separado:

- a. Utiliza un área de memoria separada para las claves en overflow.**
- b. Reduce la densidad de empaquetamiento.
- c. Utiliza una segunda función de dispersión siempre que se desee almacenar un registro en un archivo .
- d. Sólo se aplica a la dispersión extensible.

- e. Ayuda a predecir la cantidad de claves en overflow.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

52. La técnica de hash:

- a. Entorpece la inserción y el borrado de elementos.
- b. La localización de un registro siempre debe utilizar una tabla adicional en memoria.
- c. No es conveniente de aplicar sobre claves secundarias.**
- d. Requiere, al menos, de dos funciones de hash para el tratamiento de los desbordes.

53. La técnica de hash extensible:

- a. Presenta una variante de hash que permite no sólo ubicar, rápidamente, los registros, sino que, además, permite el acceso secuencial a los mismos.
- b. Siempre inserta un registro con un y sólo un acceso a disco.
- c. Siempre recupera un registro con un y sólo un acceso a disco.**
- d. En algunos casos, recupera un registro con un y sólo un acceso a disco.

54. La técnica de hashing extensible:

- a. No utiliza una función de hash porque debe cambiar dinámicamente.
- b. No utiliza una función de hash porque encuentra los registros siempre en un acceso.
- c. Utiliza una función de hash pero esta función no devuelve la dirección donde guardar el registro.**
- d. Utiliza área de desborde por separado para los registros en overflow.
- e. Todas las anteriores.
- f. a y c son correctas.
- g. b y c son correctas.
- h. a, c y d son correctas.
- i. Ninguna de las anteriores.

55. La técnica de saturación progresiva encadenada:

- a. Evita la generación de colisiones.
- b. Necesita que cada cubeta tenga capacidad para dos o más registros.
- c. Requiere, al menos, de dos funciones de hash para el tratamiento de los desbordes.
- d. Ninguna de las anteriores.**

56. Para tener un bajo porcentaje de densidad de empaquetamiento:

- a. *Se deben tener disponibles muchos más nodos de los necesarios.*
- b. Se debe utilizar una función de hash que genere pocas colisiones.
- c. Se deben tener nodos con capacidad de almacenar más de 100.000 registros.
- d. Se debe utilizar hash con espacio de direccionamiento dinámico.
- e. Todas las anteriores.
- f. Ninguna de las anteriores.

57. Se define la densidad de empaquetamiento como:

- a. El cociente entre la cantidad de registros del archivo y el espacio disponible para su almacenamiento.
- b. El cociente entre la capacidad del archivo y los registros del archivo.
- c. El cociente entre la cantidad de registros del archivo, y el producto entre la cantidad de nodos disponibles y la capacidad de estos nodos.
- d. Las respuestas a, b y c son correctas.
- e. Las respuestas a y b son correctas.
- f. Las respuestas a y c son correctas.
- g. Las respuestas b y c son correctas.

58. Se produce saturación:

- a. Siempre que dos registros diferentes obtienen de la función de hash la misma dirección de disco.
- b. Siempre que dos registros iguales obtienen de la función de hash direcciones diferentes de disco.
- c. Cuando un registro no cabe en el lugar donde debe almacenarse según el resultado de la función de hash.
- d. Cuando dos registros diferentes obtienen de la función de hash direcciones diferentes de disco.
- e. Ninguna de las anteriores.

59. Si la densidad de empaquetamiento tiende a 1 (o el 100%):

- a. Es conveniente utilizar dispersión doble para el tratamiento de overflow.
- b. Se debe cambiar la función de dispersión.
- c. Debe aumentarse el tamaño del archivo.
- d. Debe aumentarse el tamaño de los nodos.
- e. Todas las anteriores.
- f. b y c son correctas.
- g. b y d son correctas.
- h. c y d son correctas.
- i. b, c y d son correctas.
- j. Ninguna de las anteriores.

60. Si ocurrió saturación:

- a. Hubo colisión.
- b. No hubo colisión.
- c. Pudo haber ocurrido colisión.
- d. Hay más de 2 claves sinónimo.
- e. Todas las anteriores.
- f. Algunas de las anteriores.
- g. Ninguna de las anteriores.

61. Si se desea ordenar físicamente un archivo:

- a. Se puede usar hashing extensible.
- b. Se puede utilizar dispersión doble para tratar registros en saturación.
- c. Se puede utilizar saturación progresiva para tratar registros en saturación.
- d. Se puede utilizar saturación progresiva encadenada para tratar registros en saturación.
- e. La DE debe ser menor o igual a 1.
- f. Todas las anteriores.
- g. Algunas de las anteriores.
- h. Ninguna de las anteriores.

62. Si se dispone la DE de un archivo:

- a. Siempre se puede calcular la cantidad probable de registros en saturación.

- b. No se puede calcular la cantidad probable de registros en saturación.
- c. Si se calcula la cantidad probable de registros en saturación, la DE fue mayor que 1.
- d. Si se calcula la cantidad probable de registros en saturación, la DE fue menor que 1.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.**

63. Si se quiere dispersar un archivo de 40.000 elementos:

- a. Se requiere un archivo de 40.000 cubetas.
- b. Se requiere un archivo de, al menos, 40.000 cubetas.
- c. Se requiere un archivo de menos de 40.000 cubetas.
- d. Todas las anteriores.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.**

64. Si se tiene una política de hash con espacio de direccionamiento dinámico:

- a. La densidad de empaquetamiento puede ser mayor que 1.
- b. Cuando la densidad de empaquetamiento supera el 75%, se debe reacomodar al archivo.
- c. Cuando la densidad de empaquetamiento supera el 75%, se debe utilizar más espacio para nodos.
- d. Cuando la densidad de empaquetamiento supera el 75%, debe activarse una política de tratamiento de overflow, como por ejemplo área de desborde separado.
- e. Algunas de las anteriores.
- f. Ninguna de las anteriores.**

65. Un archivo tratado con *hash* estático, que tiene una densidad de empaquetamiento del 10%:

- a. *Tiene mucha fragmentación interna.*
- b. Tiene mucha fragmentación externa.
- c. *Presenta un nivel de colisiones bajo.*
- d. *Presenta un nivel de overflow bajo.*
- e. Todas las anteriores son correctas.
- f. b y c son correctas.
- g. a y c son correctas.
- h. *a, c y d son correctas.*
- i. b, c y d son correctas.
- j. Ninguna de las anteriores.

66. Una colisión:

- a. Puede no ocurrir si hay, al menos, dos claves sinónimos para una función de hashing.
- b. Puede ocurrir si hay, al menos, dos claves sinónimos para una función de hashing.
- c. *Puede utilizar un algoritmo para tratamiento de registros en saturación.*
- d. *Puede no utilizar un algoritmo para tratamiento de registros en saturación.*
- e. Todas las anteriores.
- f. *Algunas de las anteriores.*
- g. Ninguna de las anteriores.

67. Una colisión:

- a. Siempre genera saturación.
- b. Requiere usar saturación progresiva.
- c. *Puede producir saturación.*
- d. Algunas de las anteriores.
- e. Ninguna de las anteriores.

68. Una colisión se produce:

- a. *Cuando dos registros diferentes obtienen de la función de hash la misma dirección de disco.*
- b. Cuando dos registros iguales obtienen de la función de hash direcciones diferentes de disco.
- c. Cuando un registro no cabe en el lugar donde debe almacenarse de acuerdo al resultado de la función de hash.
- d. Cuando dos registros diferentes obtienen de la función de hash direcciones diferentes de disco.

69. Una función de hash:

- a. Siempre devuelve una dirección donde se debería almacenar el registro dispersado.
- b. Siempre devuelve una secuencia de bits que sirve para determinar dónde se debería almacenar el registro dispersado.
- c. *Siempre debe ser aleatoria.*
- d. Todas las anteriores.
- e. Algunas de las anteriores.

70. Una función de hash perfecta:

- a. Es difícil de conseguir.
- b. Necesita de un algoritmo para tratamiento de registros en saturación.
- c. Puede tener menos del 1% de claves sinónimos.
- d. *No tiene claves sinónimas.*
- e. Todas las anteriores.
- f. Algunas de las anteriores.